

All of the tools mentioned come with certain advantages and disadvantages. This article explores the Puppet configuration management platform, which makes the management of code and infrastructure across different stages (development, QA and production) effective. This DevOps based approach ultimately benefits organisations and their customers in getting the best out of the IT infrastructure.

An introduction to Puppet

Puppet is a leading option in the configuration management arena. It is open source and supports major operating systems such as Linux, UNIX-like OSs and Microsoft Windows. The initial release of Puppet came out in 2005. Puppet's code is written in Ruby, C++ and Clojure. The current stable release of Puppet is 6.0.4. Luke Kanies is the founder of Puppet.

Puppet consists of two major components:

- Puppet Language
- Puppet Platform

Puppet provides a modelling language, which enables the description of the resources and the end-state more simply. The Puppet language supports various platforms. The abstraction that Puppet brings in is its major advantage. To put it in simple terms, the Puppet language does not mandate expertise in programming. The configuration syntax is simple.

Declaring of resources is the major task of Puppet. The resources can be grouped into classes. For example, a resource can describe a unique file or package. Other examples of resources are as illustrated in Figure 2.

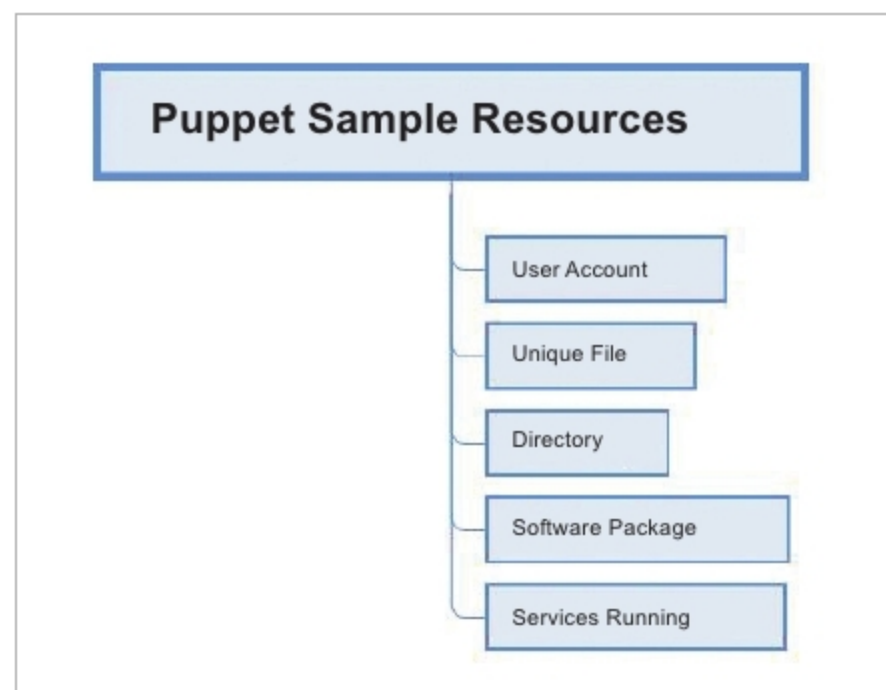


Figure 2: Puppet sample resources

A sample is shown below:

```

user { 'ksks':
  home => '/home/ksks',
  uid  => '144',
  ensure => 'present',
  comment => 'OSFY',
  shell => '/bin/bash',

```

```

groups => ['sysadmin', 'LFY', 'audio', 'video', 'puppet']
}

```

Puppet programs are termed manifests. The file name extension they are associated with is *.pp*.

Class describes an entire service or application. Puppet classes enable you to reuse the Puppet code.

```

# A class with no parameters
class base::linux {
  file { ['/etc/passwd':
    owner => 'root',
    group => 'root',
    mode  => '0644',
  ]

  file { ['/etc/shadow':
    owner => 'root',
    group => 'root',
    mode  => '0440',
  ]
}

```

The class definition incorporates the following:

- The keyword 'class'
 - The class name
 - Parameters list [optional]
 - An optional 'inherits' keyword. In this class, another class name follows.
 - Opening and closing parenthesis. Inside this block, the Puppet code with at least one resource declaration.
- The class definition can include parameters as well:

```

# A class with parameters
class apache (String $version = 'latest') {
  package {'httpd':
    ensure => $version, # Using the class parameter from
above
    before => File['/etc/httpd.conf'],
  }
  file {'/etc/httpd.conf':
    ensure => file,
    owner  => 'httpd',
    content => template('apache/httpd.conf.erb'), # Template
from a module
  }

  service {'httpd':
    ensure  => running,
    enable  => true,
    subscribe => File['/etc/httpd.conf'],
  }
}

```